PRACTICAL-7

1. MST USING PRIMS'S APPROACH.

Answer:

```
import java.util.Scanner;

public class Main {

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print(&quot;Enter number of nodes: &quot;);
int n = sc.nextInt();

int[][] g = new int[n][n];
int[][] stree = new int[n][3];

for (int i = 0; i &lt; n; i++) {
stree[i][0] = stree[i][1] = stree[i][2] = -1;
}

System.out.println(&quot;Enter graph edges (0 for no edge
and weight for edge):&quot;);
char c = &#39;a&#39;;
for (int i = 0; i &lt; n; i++) {
for (int j = i; j &lt; n; j++) {
g[i][j] = 0;
if (i != j) {
System.out.print(&quot;(&quot; + (char) (c + i) + &quot;,&quot; + (char) (c
+ j) + &quot;):-&quot;);
g[i][j] = sc.nextInt(); // Corrected: Input read on
```

```
the same line
g[j][i] = g[i][j];
}
}
}

// ... (rest of the code remains the same)
System.out.print(&quot;\t &quot;);
for (int i = 0; i &lt; n; i++) {
System.out.print((char) (c + i) + &quot;\t&quot;);
}
System.out.println();

for (int i = 0; i &lt; n; i++) {
System.out.print((char) (c + i) + &quot;\t &quot;);
for (int j = 0; j &lt; n; j++) {
if (g[i][j] != 0) {
System.out.print(&quot;|&quot; + g[i][j]);
} else {
System.out.print(&quot;|-&quot;);
}
System.out.print(&quot;\t&quot;);
}
System.out.println();
}

// Root node

stree[0][0] = stree[0][1] = 0;
stree[0][2] = 1;

// Making stree
int counter = 0, i = 0;
while (counter &lt; n) {
```

```
for (int k = 0; k &lt; n; k++) {
if (g[i][k] != 0) {
if (stree[k][0] == -1 &amp;&amp; stree[k][2] != 1) {
stree[k][0] = i;
stree[k][1] = g[i][k];
} else {
if (stree[k][1] &gt; g[i][k] &amp;&amp; stree[k][2] != 1) {
stree[k][0] = i;
stree[k][1] = g[i][k];
}
}
}
}

int temp = -1;
for (int k = 0; k &lt; n; k++) {

if (stree[k][2] != 1 &amp;&amp; stree[k][0] != -1) {
if (temp == -1) {
temp = k;
} else {
if (stree[temp][1] &gt; stree[k][1]) {
temp = k;
}
}
}
}
i = temp;
stree[i][2] = 1;
counter++;
}

for (i = 0; i &lt; n; i++) {
System.out.println((char) (c + i) + &quot;:-&quot; + (char) (c +
```

```
stree[i][0]) + &quot;,&quot; + stree[i][1]);
}
sc.close();
}
}
```

Screenshot of Output:

```
Array size: 100, Average time: 0.00213621 ms
Array size: 1000, Average time: 9.068300000000001E-4 ms
Array size: 10000, Average time: 0.00480351 ms
Array size: 100000, Average time: 5.569700000000001E-4 ms
```

PRACTICAL-8

2. MST USING KRUSKA'S APPROACH.

Answer:

```
import java.util.Scanner;
import java.util.Arrays;

public class KruskalMST {

static class Edge { // Inner class to avoid separate file
issues
int w, e1, e2;
Edge(int w, int e1, int e2) {
this.w = w;
this.e1 = e1;
this.e2 = e2;
}
}
```

```
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
char c = &#39;a&#39;;
int k = 0;

System.out.print(&quot;Enter number of nodes: &quot;);
int n = scanner.nextInt();
int[][] g = new int[n][n];
int[][] stree = new int[n][n];
int[] snode = new int[n];
Edge[] edges = new Edge[n * n];

// Initialize tree matrix
for (int i = 0; i &lt; n; i++) Arrays.fill(stree[i], -1);

System.out.println(&quot;Enter graph edges (0 for no edge,
weight for edge):&quot;);
for (int i = 0; i &lt; n; i++) {
for (int j = i; j &lt; n; j++) {
g[i][j] = 0;
if (i != j) {
System.out.print(&quot;(&quot; + (char) (c + i) + &quot;,&quot; + (char) (c
+ j) + &quot;):- &quot;);
g[i][j] = scanner.nextInt();
g[j][i] = g[i][j];
if (g[i][j] != 0) {
edges[k++] = new Edge(g[i][j], i, j);
}
}
}
}

// Sort edges by weight
```

```
Arrays.sort(edges, 0, k, (e1, e2) -&gt; Integer.compare(e1.w,
e2.w));

// Initialize graph-tree table
Arrays.fill(snode, -1);
for (int i = 0; i &lt; n; i++) Arrays.fill(stree[i], -1);

int gp = 0;
for (int i = 0; i &lt; k; i++) {
if (snode[edges[i].e1] == -1 &amp;&amp; snode[edges[i].e2] == -
1) {
stree[edges[i].e1][edges[i].e2] =
stree[edges[i].e2][edges[i].e1] = edges[i].w;
snode[edges[i].e1] = snode[edges[i].e2] = gp++;
} else if (stree[edges[i].e1][edges[i].e2] == -1 &amp;&amp;
snode[edges[i].e1] != snode[edges[i].e2]) {
stree[edges[i].e1][edges[i].e2] =
stree[edges[i].e2][edges[i].e1] = edges[i].w;
if (snode[edges[i].e1] != -1 &amp;&amp; snode[edges[i].e2] ==
-1) {
snode[edges[i].e2] = snode[edges[i].e1];
} else if (snode[edges[i].e2] != -1 &amp;&amp;
snode[edges[i].e1] == -1) {
snode[edges[i].e1] = snode[edges[i].e2];
} else {

int temp = snode[edges[i].e2];
for (int l = 0; l &lt; n; l++) {
if (snode[l] == temp) {
snode[l] = snode[edges[i].e1];
}
}
}
}
```

```
}

// Printing the final MST table
System.out.print(&quot;\n\t &quot;);
for (int i = 0; i &lt; n; i++) System.out.print((char) (c + i) + &quot;\t
&quot;);
for (int i = 0; i &lt; n; i++) {
System.out.println();
System.out.print((char) (c + i) + &quot;\t&quot;);
for (int j = 0; j &lt; n; j++) {
if (stree[i][j] != -1) System.out.print(&quot;|&quot; + stree[i][j] +
&quot;\t&quot;);
else System.out.print(&quot;|-\t&quot;);
}

}
scanner.close();
}
}
```

Screenshot of Output:

```
Bubble Sort Time: 0.00604719 seconds
Selection Sort Time: 0.00422289 seconds
Insertion Sort Time: 0.004044771 seconds
```

PRACTICAL-9

3. BFS-SPANNING.

Answer:

```
import java.util.*;

public class BFS {
static int[][] adjMatrix;
static boolean[] visited;
static int n;

public static void bfs(int start) {
Queue&lt;Integer&gt; queue = new LinkedList&lt;&gt;();
queue.add(start);
visited[start] = true;

System.out.println(&quot;BFS Traversal:&quot;);
while (!queue.isEmpty()) {

int node = queue.poll();
System.out.print(node + &quot; &quot;);

for (int i = 1; i &lt;= n; i++) {
if (adjMatrix[node][i] == 1 &amp;&amp; !visited[i]) {
queue.add(i);
visited[i] = true;
}
}
}
}

public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);

System.out.print(&quot;Enter number of vertices: &quot;);
n = scanner.nextInt();

adjMatrix = new int[n + 1][n + 1]; // Adjust for 1-based
```

```
indexing
visited = new boolean[n + 1];

System.out.println(&quot;Enter adjacency matrix:&quot;);
for (int i = 1; i &lt;= n; i++) {
for (int j = 1; j &lt;= n; j++) {
adjMatrix[i][j] = scanner.nextInt();
}
}

System.out.print(&quot;Enter starting node: &quot;);
int start = scanner.nextInt();

bfs(start);
scanner.close();
}
}
```

Screenshot of Output:

```
Enter a number to find factorial: 14

Time for iterative is 2e-06

Factorial of 14 is 8.71783e+10

Time for recursive is 1e-06

Factorial of 14 is -2.14748e+09
```

PRACTICAL-10

4. **DFS-SPANNING.**

Answer:

```
import java.util.LinkedList;

import java.util.Queue;
import java.util.Scanner;

public class BFS {
static int[][] a = new int[20][20]; // Adjacency matrix
static boolean[] reach = new boolean[20]; // Visited nodes
static int n; // Number of vertices

static void bfs(int v) {
Queue&lt;Integer&gt; queue = new LinkedList&lt;&gt;();
queue.add(v);
reach[v] = true;
```

```
while (!queue.isEmpty()) {
int node = queue.poll(); // Dequeue

for (int i = 1; i &lt;= n; i++) {
if (a[node][i] == 1 &amp;&amp; !reach[i]) {
System.out.println(node + &quot; -&gt; &quot; + i);
queue.add(i); // Enqueue
reach[i] = true;
}

}
}
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print(&quot;\nEnter number of vertices: &quot;);
n = sc.nextInt();

// Initialize reach array
for (int i = 1; i &lt;= n; i++) {
reach[i] = false;
for (int j = 1; j &lt;= n; j++) {
a[i][j] = 0;
}
}

System.out.println(&quot;\nEnter adjacency matrix:&quot;);
for (int i = 1; i &lt;= n; i++) {
for (int j = 1; j &lt;= n; j++) {
a[i][j] = sc.nextInt();

}
```

```
}

bfs(1); // Start BFS from node 1

// Check if all nodes are visited
boolean connected = true;
for (int i = 1; i &lt;= n; i++) {
if (!reach[i]) {
connected = false;
break;
}
}

if (connected)
System.out.println(&quot;\nGraph is connected&quot;);
else
System.out.println(&quot;\nGraph is not connected&quot;);

sc.close();
}
}
```

Screenshot of Output:

```
Enter size of array: 4
Enter 4 elements in array:
3
54
5
3

Sorted heap is:
3 3 5 54



        54

      5 3

    3
```

PRACTICAL-11

5.DIJASTRA'S IN MST.

Answer:

```
import java.util.Scanner;

public class KaratsubaMultiplication {

// Function to perform Karatsuba multiplication
public static long karatsuba(long x, long y) {
if (x &lt; 10 || y &lt; 10) // Base case: when x or y is a single
digit
return x * y;

int n = Math.max(String.valueOf(x).length(),
String.valueOf(y).length());
int half = n / 2;

long power = (long) Math.pow(10, half);
long a = x / power; // First half of x
long b = x % power; // Second half of x
long c = y / power; // First half of y
long d = y % power; // Second half of y

long ac = karatsuba(a, c);
long bd = karatsuba(b, d);
long ad_plus_bc = karatsuba(a + b, c + d) - ac - bd;

return ac * (long) Math.pow(10, 2 * half) + (ad_plus_bc *
power) + bd;
```

```
}

public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);

System.out.print(&quot;Enter first number: &quot;);
long x = scanner.nextLong();

System.out.print(&quot;Enter second number: &quot;);
long y = scanner.nextLong();

long result = karatsuba(x, y);
System.out.println(&quot;Karatsuba Multiplication Result: &quot; +
result);

scanner.close();
}
}
```

Screenshot of Output:

```
Enter number of objects: 5
Enter maximum weight, knapsack can carry: 3
Enter weights of objects:
w[1] = 4
w[2] = 5
w[3] = 6
w[4] = 5
w[5] = 4
Enter profits/values of objects:
v[1] = 3
v[2] = 5
v[3] = 6
v[4] = 4
v[5] = 7

P    W    i
0    1    2    3
0    0    0    0    0    0    0
3    4    1    0    0    0    0
5    5    2    0    0    0    0
6    6    3    0    0    0    0
4    5    4    0    0    0    0
7    4    5    0    0    0    0
```

PRACTICAL-12

6. MODULAR EXPONENTIAL IN DIVIDE AND CONQUER.

Answer:

```
import java.util.Scanner;

public class ModularExponentiation {
// Function to compute (x^y) % p using iterative method
static long modExpo(long x, long y, long p) {
long result = 1; // Initialize result

x = x % p; // Ensure x is within modulo range

if (x == 0) return 0; // If x is divisible by p, result is 0
```

```
while (y &gt; 0) {
if ((y &amp; 1) == 1) // If y is odd, multiply x with result
result = (result * x) % p;

y = y &gt;&gt; 1; // Divide y by 2
x = (x * x) % p; // Square x and apply modulo
}
return result;
}

// Driver Code
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);

System.out.print(&quot;Enter base (x): &quot;);
long x = scanner.nextLong();

System.out.print(&quot;Enter exponent (y): &quot;);

long y = scanner.nextLong();

System.out.print(&quot;Enter modulus (p): &quot;);
long p = scanner.nextLong();

System.out.println(&quot;Result: &quot; + modExpo(x, y, p));

scanner.close();
}
}
```

Screenshot of Output:

```
Enter number of matrices: 2
Enter P array values:
p[0] = 2
p[1] = 3
p[2] = 4


m Table


0   24
0   0


s Table


0   1
0   0


The solution is:
(A1A2)
```